

Nombre: Luis de la Garza González

Matrícula: al03101869

Nombre del curso: Machine Learning.

Trabajo: Actividad 02.- Las Matrices y sus Aplicaciones.

Nombre del profesor: Zuriel Dathan Mora Felix

Fecha: 21 de septiembre de 2025

Contenido

Objetivo	3
Instrucciones:	3
Desarrollo	5
1. Investigación sobre normas.....	5
2. Operaciones utilizando Jupyter Notebook y Python.	7
3. Coordenada λ lambda, proyección y matriz de proyección.	11
4. Operaciones con matrices cuadradas.....	18
5. Cálculo de autovalores y autovectores.....	21
Conclusiones	24
Liga al código en Github	25
Anexo.- Infografía sobre Tipos de Modelos Vectoriales y sus aplicaciones	25

Objetivo

Explorar conceptos fundamentales de álgebra lineal aplicados a vectores, normas y proyecciones, utilizando Python y Jupyter Notebook.

Instrucciones:

1. Investigación sobre normas

- Investiga los diferentes tipos de normas que existen.
- Identifica sus principales aplicaciones.
- Elabora una infografía para compartir lo aprendido.

2. Utilizando un cuaderno de Jupyter Notebook y el lenguaje de programación Python, realiza las siguientes operaciones:

- Genera dos vectores en $\mathbb{R}^2$ y represéntalos en un plano utilizando la librería Matplotlib.
- Calcula el producto punto entre los dos vectores generados.
- Calcula la longitud de ambos vectores.
- Encuentra el ángulo que forman los dos vectores entre sí y genera un nuevo vector que sea ortogonal a cualquiera de los dos primeros.

3. Sea el vector base $\mathbf{b} = \begin{bmatrix} 5 \\ 3 \end{bmatrix} \in \mathbb{R}^2$, que genera el subespacio unidimensional $\mathcal{U} \subseteq \mathbb{R}^2$.

Encuentra:

- El valor de la coordenada λ lambda
- La proyección $\pi_{\mathcal{U}}(\mathbf{x}) \in \mathcal{U}$
- La matriz de proyección P_{π} para proyectar por el subespacio $\mathcal{U}$ el vector $\mathbf{x} = \begin{bmatrix} -2 \\ 7 \end{bmatrix} \in \mathbb{R}^2$.
Utiliza un cuaderno de Jupyter Notebook para elaborar tu respuesta.

4. Operaciones con matrices cuadradas

Usando Python en un nuevo cuaderno de Jupyter Notebook:

- Genera una matriz cuadrada $\mathbb{R}^{n \times n}$ con $(n > 3)$ y calcula su determinante y su traza.

- Calcula el rango de la matriz generada y los valores de los coeficientes del polinomio característico.
- Encuentra las raíces del polinomio característico de la matriz generada

5. Calcula los autovalores y autovectores de la siguiente matriz:

$$A = \begin{bmatrix} 5, 1 \\ 2, 2 \end{bmatrix}$$

realiza su autodescomposición y grafica los resultados obtenidos en un plano con Matplotlib.

Desarrollo

1. Investigación sobre normas

- Investiga los diferentes tipos de normas que existen.
- Identifica sus principales aplicaciones.

En álgebra lineal, las normas son funciones que asignan un número real no negativo a un vector, proporcionando una medida de su "tamaño" o "longitud". Existen varios tipos de normas, cada una con sus propias características y aplicaciones:

Norma	Principales Aplicaciones
Norma Euclidiana (Norma (L_2)): También conocida como la norma (L_2) o norma (2), es la más común y se define como la raíz cuadrada de la suma de los cuadrados de las componentes del vector. Para un vector $(x = (x_1, x_2, \dots, x_n))$, se calcula como: $x _2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$	Ampliamente utilizada en la geometría y en la teoría de la optimización. Es fundamental en el cálculo de distancias en el espacio euclidiano, lo que la hace esencial en algoritmos de aprendizaje automático, como el k-means clustering y los métodos de regresión. También se utiliza en la física para calcular magnitudes como la velocidad y la aceleración.
Norma Manhattan (Norma (L_1)): También conocida como la norma (L_1) o norma de la "suma absoluta", se define como la suma de los valores absolutos de las componentes del vector: $[x _1 = x_1 + x_2 + \dots + x_n]$	se utiliza en problemas de optimización donde se desea minimizar la suma de las diferencias absolutas. Es común en la teoría de redes y en la optimización de rutas, como en el problema del viajante. También se utiliza en el aprendizaje automático, especialmente en técnicas de regularización como la regresión Lasso, que ayuda a seleccionar características importantes en modelos predictivos.
Norma Infinita (Norma (L_∞)): También conocida como la norma del "máximo", se define como el valor absoluto máximo de las componentes del vector: $[x _\infty = \max(x_1 , x_2 , \dots, x_n)]$	se utiliza en problemas de optimización donde se desea minimizar el error máximo. Es común en el análisis de estabilidad de sistemas y en el diseño de controladores robustos. También se aplica en la teoría de juegos y en la programación lineal, donde se busca minimizar el peor caso posible.

Norma	Principales Aplicaciones
Norma (L_p): Es una generalización de las normas (L_1), (L_2) y (L_{∞}). Para un valor ($p \geq 1$), la norma (L_p) se define como: $ x _p = (x_1 ^p + x_2 ^p + \dots + x_n ^p)^{1/p}$ Cuando ($p = 2$), se obtiene la norma Euclidiana, y cuando ($p = 1$), se obtiene la norma Manhattan.	Las normas (L_p) son una generalización de las normas (L_1), (L_2) y (L_{∞}), y se utilizan en diversas aplicaciones de análisis funcional y teoría de espacios de Banach. En el procesamiento de señales, las normas (L_p) se utilizan para medir la energía de una señal y en la compresión de datos. También se aplican en la teoría de la probabilidad y en el análisis de datos.
Norma de Frobenius: Es una norma utilizada para matrices y se define como la raíz cuadrada de la suma de los cuadrados de todos los elementos de la matriz. Para una matriz (A), se calcula como: $ A _F = \sqrt{\sum_{i,j} a_{ij} ^2}$	Se utiliza principalmente en el análisis de matrices. Es útil en la descomposición en valores singulares (SVD) y en la factorización de matrices, que son técnicas fundamentales en el procesamiento de imágenes y en la reducción de dimensionalidad. También se utiliza en la teoría de control y en la optimización de sistemas dinámicos.

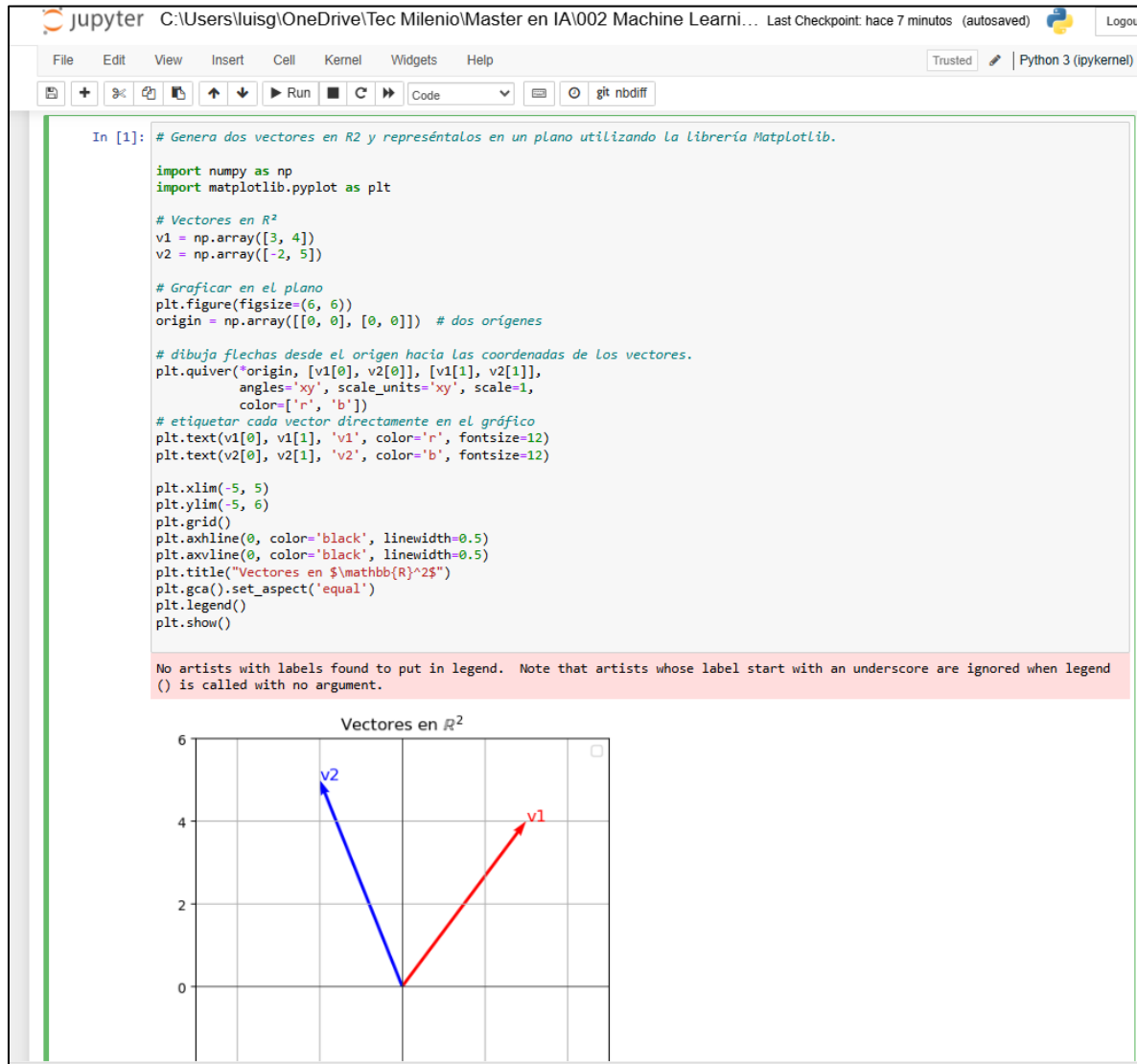
c) Elabora una infografía para compartir lo aprendido.

Ver [anexo](#) o abrir archivo de Power Point “Infografía sobre Tipos de Modelos Vectoriales y sus aplicaciones.pptx”

2. Operaciones utilizando Jupyter Notebook y Python.

Utilizando un cuaderno de Jupyter Notebook y el lenguaje de programación Python, realiza las siguientes operaciones:

- Genera dos vectores en $\mathbb{R}^2$ y represéntalos en un plano utilizando la librería Matplotlib.



- Calcula el producto punto entre los dos vectores generados.

Para calcular el producto punto entre dos vectores, utilizamos la siguiente fórmula:

$$a \cdot b = a_1b_1 + a_2b_2 + \dots + a_nb_n$$

el producto punto se calcula de la siguiente manera:

$$v_1 \cdot v_2 = (3)(-2) + (4)(5) = -6 + 20 = 14$$

```
In [2]: import numpy as np

a = np.array([3, 4])
b = np.array([-2, 5])

producto_punto = np.dot(a, b)
print("Producto punto:", producto_punto)
```

Producto punto: 14

- Calcula la longitud de ambos vectores.

Para calcular la **longitud** (o norma) de un vector en R^2 , usamos la fórmula:

$$||\vec{v}|| = (x^2 + y^2)^{1/2}$$

```
In [3]: import numpy as np  
  
v1 = np.array([3, 4])  
v2 = np.array([-2, 5])  
  
norm_v1 = np.linalg.norm(v1)  
norm_v2 = np.linalg.norm(v2)  
  
print("Longitud de v1:", norm_v1)  
print("Longitud de v2:", norm_v2)
```

```
Longitud de v1: 5.0  
Longitud de v2: 5.385164807134504
```

- Encuentra el ángulo que forman los dos vectores entre sí y genera un nuevo vector que sea ortogonal a cualquiera de los dos primeros.

Ángulo que forman los dos vectores entre sí:

```
In [4]: import numpy as np

v1 = np.array([3, 4])
v2 = np.array([-2, 5])

# Producto punto
dot_product = np.dot(v1, v2)

# Normas
norm_v1 = np.linalg.norm(v1)
norm_v2 = np.linalg.norm(v2)

# Ángulo en radianes
theta_rad = np.arccos(dot_product / (norm_v1 * norm_v2))

# Ángulo en grados
theta_deg = np.degrees(theta_rad)

print("Ángulo en radianes:", theta_rad)
print("Ángulo en grados:", theta_deg)
```

Ángulo en radianes: 1.0240074859056492
Ángulo en grados: 58.67130713219583

- **Nuevo vector que sea ortogonal a cualquiera de los dos primeros:**

Una forma rápida de encontrar un vector ortogonal en R^2 es rotarlo 90° : $v1_{\perp} = [-y, x] = [-4, 3]$

Verificación: el producto punto de dos vectores ortogonales debe ser cero:

$$v1 \cdot v_{\perp} = 3 \cdot (-4) + 4 \cdot 3 = -12 + 12 = 0$$

```
In [5]: v1 = np.array([3, 4])
v_ortogonal = np.array([-4, 3])

print("Vector ortogonal a v1:", v_ortogonal)
print("Producto punto:", np.dot(v1, v_ortogonal)) # Debe ser 0
```

Vector ortogonal a v1: [-4 3]
Producto punto: 0

3. Coordenada λ lambda, proyección y matriz de proyección.

Sea el vector base $\vec{u}$ el vector $\vec{x} = \begin{bmatrix} -2 \\ 7 \end{bmatrix} \in \mathbb{R}^2$ que genera el subespacio $\mathcal{U} \subseteq \mathbb{R}^2$ unidimensional

Encuentra:

- El valor de la coordenada λ lambda

Interpretación geométrica

El valor de λ indica cuánto se escala el vector base para llegar al vector $\vec{x}$. Si ambos componentes coinciden, entonces

$\vec{x}$ pertenece al subespacio generado por $\vec{v}$

Utiliza un cuaderno de Jupyter Notebook para elaborar tu respuesta.

```
In [6]: # Sea el vector base [-2, 7] que genera el subespacio unidimensional Encuentra:
# El valor de la coordenada  $\lambda$  lambda

v_base = np.array([-2, 7])
x = np.array([-4, 14])

# Verificamos si x es múltiplo de v_base
lambda_val = x[0] / v_base[0] # suponiendo v_base[0]  $\neq 0$ 

# Confirmamos con el segundo componente
if np.isclose(x[1], lambda_val * v_base[1]):
    print(" $\lambda$  =", lambda_val)
else:
    print("El vector no pertenece al subespacio generado por v_base.")

 $\lambda = 2.0$ 
```

- La proyección $\pi_u(x) \in U$

```

jupyter C:\Users\luisg\OneDrive\Tec Milenio\Master en IA\002 Machine Le... Last Checkpoint: hace unos segundos (autosaved) Python 3 (ipykernel)
File Edit View Insert Cell Kernel Widgets Help Trusted
In [9]: # Proyección ortogonal de un vector sobre un subespacio
        # Supongamos que: u = [-2, 7] y x = [3, 1]

        import numpy as np

        u = np.array([-2, 7])
        x = np.array([3, 1])

        # Proyección ortogonal de x sobre el subespacio generado por u
        projection = (np.dot(x, u) / np.dot(u, u)) * u
        print("π_U(x) =", projection)

        #####
        # Visualización de la proyección ortogonal
        import numpy as np
        import matplotlib.pyplot as plt

        # Vector base del subespacio
        u = np.array([-2, 7])
        # Vector a proyectar
        x = np.array([3, 1])

        # Cálculo de la proyección ortogonal
        lambda_proj = np.dot(x, u) / np.dot(u, u)
        projection = lambda_proj * u

        # Graficamos
        origin = np.array([0, 0])
        fig, ax = plt.subplots()

        # Vector original x
        ax.quiver(*origin.T, x[0], x[1], angles='xy', scale_units='xy', scale=1, color='blue', label='x')

        # Vector base u
        ax.quiver(*origin.T, u[0], u[1], angles='xy', scale_units='xy', scale=1, color='green', label='u')

        # Proyección π_U(x)
        ax.quiver(*origin.T, projection[0], projection[1], angles='xy', scale_units='xy', scale=1, color='red', label='π_U(x)')

        # Línea desde x hasta su proyección
        ax.plot([x[0], projection[0]], [x[1], projection[1]], 'k--', linewidth=1)

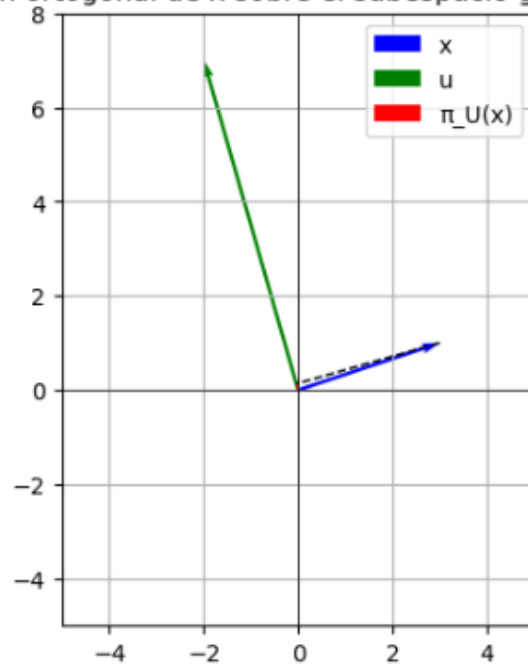
        # Configuración del gráfico
        ax.set_xlim(-5, 5)
        ax.set_ylim(-5, 8)
        ax.set_aspect('equal')
        ax.grid(True)
        ax.axhline(0, color='black', linewidth=0.5)
        ax.axvline(0, color='black', linewidth=0.5)
        ax.set_title("Proyección ortogonal de x sobre el subespacio generado por u")
        ax.legend()
        plt.show()

π_U(x) = [-0.03773585  0.13207547]

```

$$\pi_U(x) = [-0.03773585 \quad 0.13207547]$$

Proyección ortogonal de x sobre el subespacio generado por u



- La matriz de proyección P_π para proyectar por el subespacio

La **matriz de proyección ortogonal** P_π permite proyectar cualquier vector $\vec{x} \in \mathbb{R}^n$ sobre el subespacio generado por un vector base u , es una herramienta clave para entender cómo se descompone un vector en componentes alineadas y ortogonales al subespacio.

```
In [10]: # matriz de proyección P $\pi$  para proyectar por el subespacio

import numpy as np

u = np.array([-2, 7]).reshape(-1, 1) # vector columna
x = np.array([3, 1]).reshape(-1, 1)

# Matriz de proyección
P_pi = (u @ u.T) / (u.T @ u)

# ahora encontramos la proyección del vector x sobre el vector u multiplicando la matriz de proyección por el vector x
projection = P_pi @ x

print("Matriz de proyección P $\pi$ :\n", P_pi)
print("P $\pi$ (x):\n", projection.flatten())

Matriz de proyección P $\pi$ :
[[ 0.0754717 -0.26415094]
 [-0.26415094  0.9245283 ]]
P $\pi$ (x):
[-0.03773585  0.13207547]
```

Para encontrar el ángulo que forman los dos vectores (v_1) y (v_2) , utilizamos la fórmula del producto punto y la magnitud de los vectores. La fórmula es:

$$\cos(\theta) = \frac{\vec{v}_1 \cdot \vec{v}_2}{\|\vec{v}_1\| \cdot \|\vec{v}_2\|}$$

Donde (θ) es el ángulo entre los dos vectores.

d) Calcula los autovalores y autovectores de la siguiente matriz $A = \begin{bmatrix} 5 & 1 \\ 2 & 2 \end{bmatrix}$.

```

jupyter C:\Users\luig\OneDrive\Tec Milenio\Master en IA\002 Machine Le... Last Checkpoint: hace unos segundos (autosaved) Logout
File Edit View Insert Cell Kernel Widgets Help Trusted Python 3 (ipykernel)
In [1]: # Cálculo de autovalores y autovectores

import numpy as np

# Definimos la matriz
A = np.array([[5, 1],
              [2, 2]])

# Calculamos autovalores y autovectores
eigenvalues, eigenvectors = np.linalg.eig(A)

print("Autovalores:", eigenvalues)
print("Autovectores:\n", eigenvectors)

Autovalores: [5.56155281 1.43844719]
Autovectores:
[[ 0.87192821 -0.27032301]
 [ 0.48963374  0.96276969]]

```

Realiza su autodescomposición y grafica los resultados obtenidos en un plano con Matplotlib.

```

In [1]: # Realiza su autodescomposición y grafica los resultados obtenidos en un plano con Matplotlib.

import numpy as np
import matplotlib.pyplot as plt

# Matriz A
A = np.array([[5, 1], [2, 2]])

# Autovalores y autovectores
eigenvalues, eigenvectors = np.linalg.eig(A)

# Origen para cada flecha
origin = np.zeros((2, eigenvectors.shape[1]))

# Crear figura
plt.figure(figsize=(6, 6))

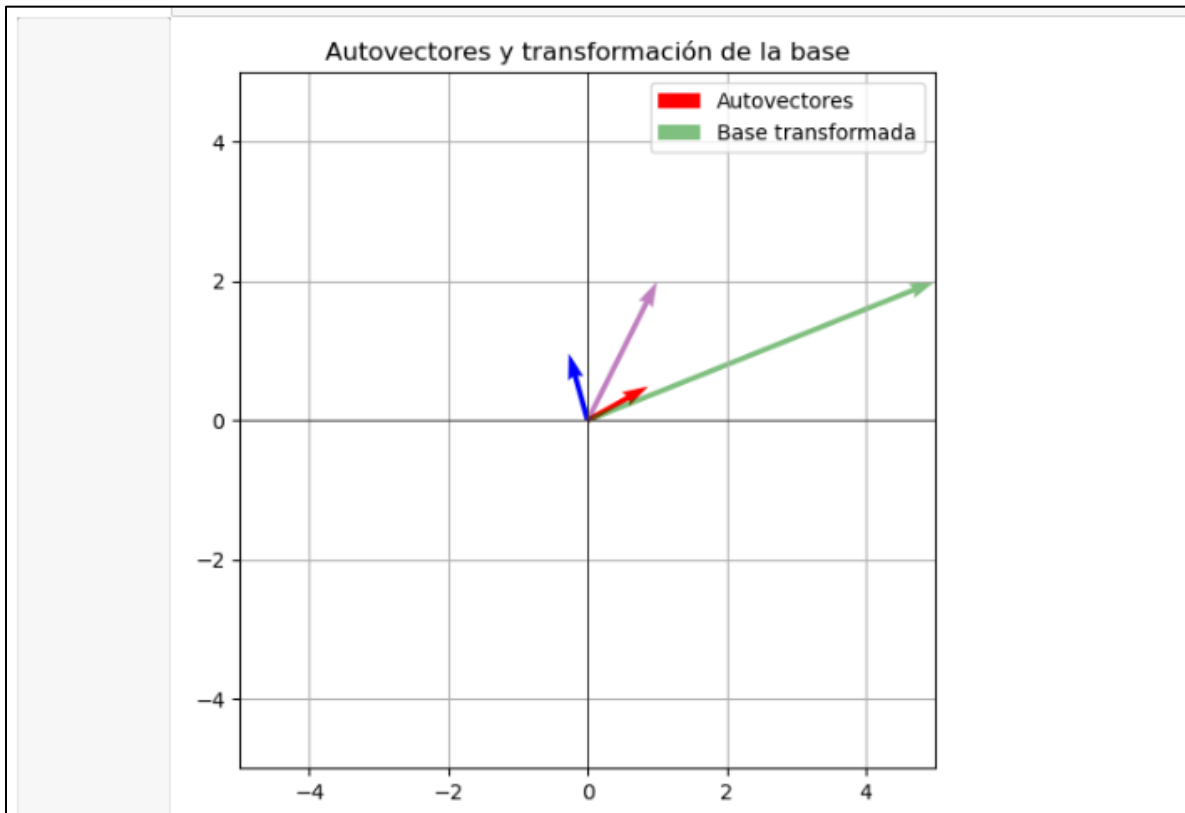
# Graficar autovectores
plt.quiver(*origin, eigenvectors[0], eigenvectors[1],
          angles='xy', scale_units='xy', scale=1,
          color=['r', 'b'], label='Autovectores')

# Graficar transformación de la base estándar
x = np.array([[1, 0], [0, 1]])
transformed = A @ x.T
plt.quiver(*origin, transformed[0], transformed[1],
          angles='xy', scale_units='xy', scale=1,
          color=['g', 'purple'], alpha=0.5, label='Base transformada')

# Estética del gráfico
plt.xlim(-5, 5)
plt.ylim(-5, 5)
plt.grid()
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.title("Autovectores y transformación de la base")
plt.gca().set_aspect('equal')
plt.legend()

# Guardar imagen
plt.savefig("autovectores_matriz_A.png", dpi=300)
plt.show()

```


**Interpretación:**

Las flechas rojas y azules representan los autovectores, que mantienen su dirección bajo la acción de A .

Las flechas verdes y moradas muestran cómo la base estándar se transforma, evidenciando el efecto de la matriz sobre el espacio.

Esta visualización confirma que los autovectores son direcciones privilegiadas donde la transformación solo escala, sin rotar.

4. Operaciones con matrices cuadradas

Usando Python en un nuevo cuaderno de Jupyter Notebook:

- Genera una matriz cuadrada $\mathbb{R}^{n \times n}$ con ($n > 3$) y calcula su determinante y su traza.

```
In [11]: # Genera una matriz cuadrada con ( n > 3 ) y calcula su determinante y su traza.

import numpy as np

# Tamaño de la matriz (n > 3)
n = 4

# Generar matriz cuadrada aleatoria
A = np.random.randint(-10, 10, size=(n, n))
print("Matriz A:\n", A)

# Calcular determinante
det_A = np.linalg.det(A)
print("\nDeterminante de A:", det_A)

# Si el determinante es distinto de cero, la matriz es invertible, lo que implica que la transformación es reversible.
# El determinante permite saber si un sistema de ecuaciones tiene solución única (si es distinto de cero).

# Calcular traza (suma de la diagonal)
trace_A = np.trace(A)
print("Traza de A:", trace_A)

Matriz A:
[[ 8  4 -7 -3]
 [ 5 -3  4  6]
 [-1 -3  4  8]
 [-3 -4 -4 -9]]

Determinante de A: -2267.9999999999995
Traza de A: 0
```

- Calcula el rango de la matriz generada y los valores de los coeficientes del polinomio característico.

El rango de una matriz es el número máximo de filas (o columnas) linealmente independientes. Para encontrar el rango, podemos reducir la matriz a su forma escalonada mediante eliminación gaussiana.

```
In [12]: rango = np.linalg.matrix_rank(A)
print("Rango de A:", rango)
```

Rango de A: 4

```
In [ ]: # coeficientes del polinomio característico.  
coeficientes = np.poly(A)  
print("Coeficientes del polinomio característico:", coeficientes)
```

El polinomio característico se define como:

$$p(\lambda) = \det(A - \lambda I)$$

donde (I) es la matriz identidad y (λ) es un escalar.

Para obtener sus coeficientes:

```
In [13]: # coeficientes del polinomio característico.
coeficientes = np.poly(A)
print("Coeficientes del polinomio característico:", coeficientes)

Coeficientes del polinomio característico: [ 1.00000000e+00 -1.02140518e-14 -5.30000000e+01 -7.28000000e+02
-2.26800000e+03]
```

- Encuentra las raíces del polinomio característico de la matriz generada

```
In [14]: # Raíces del polinomio característico

import numpy as np

coef = [1.0, -1.02140518e-14, -53.0, -728.0, -2268.0]
raices = np.roots(coef)

print("Raíces del polinomio:", raices)

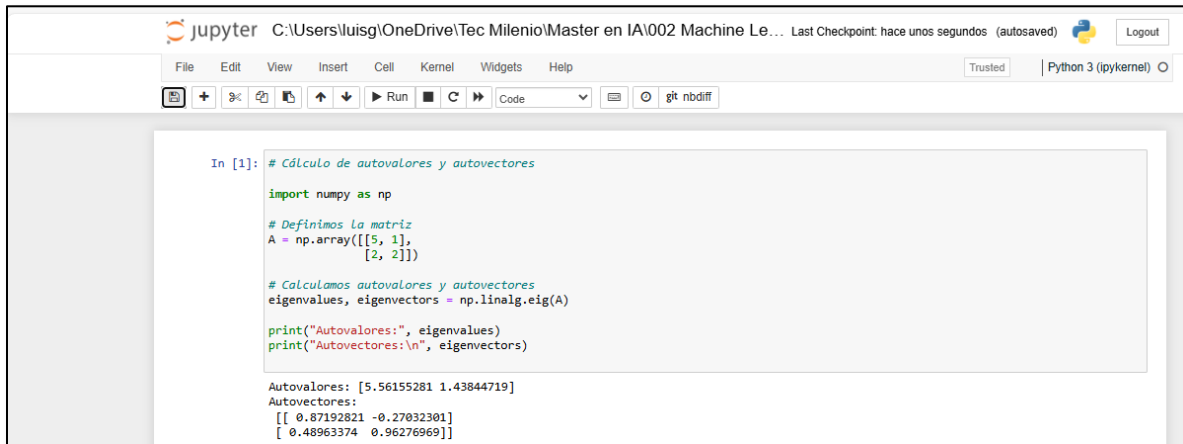
Raíces del polinomio: [11.53822148+0.j          -3.81586338+5.97966899j -3.81586338-5.97966899j
-3.90649472+0.j          ]
```

Esto indica que la matriz tiene una mezcla de modos reales y oscilatorios. En contextos como sistemas dinámicos o machine learning, esto puede reflejar componentes estables y rotacionales.

5. Cálculo de autovalores y autovectores.

Calcula los autovalores y autovectores de la siguiente matriz

$$A = \begin{bmatrix} 5 & 1 \\ 2 & 2 \end{bmatrix},$$



```
In [1]: # Cálculo de autovalores y autovectores

import numpy as np

# Definimos la matriz
A = np.array([[5, 1],
              [2, 2]])

# Calculamos autovalores y autovectores
eigenvalues, eigenvectors = np.linalg.eig(A)

print("Autovalores:", eigenvalues)
print("Autovectores:\n", eigenvectors)

Autovalores: [5.56155281 1.43844719]
Autovectores:
[[ 0.87192821 -0.27032301]
 [ 0.48963374  0.96276969]]
```

Realiza su autodescomposición y grafica los resultados obtenidos en un plano con Matplotlib.

In [1]: # Realiza su autodescomposición y grafica los resultados obtenidos en un plano con Matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

# Matriz A
A = np.array([[5, 1], [2, 2]])

# Autovalores y autovectores
eigenvalues, eigenvectors = np.linalg.eig(A)

# Origen para cada flecha
origin = np.zeros((2, eigenvectors.shape[1]))

# Crear figura
plt.figure(figsize=(6, 6))

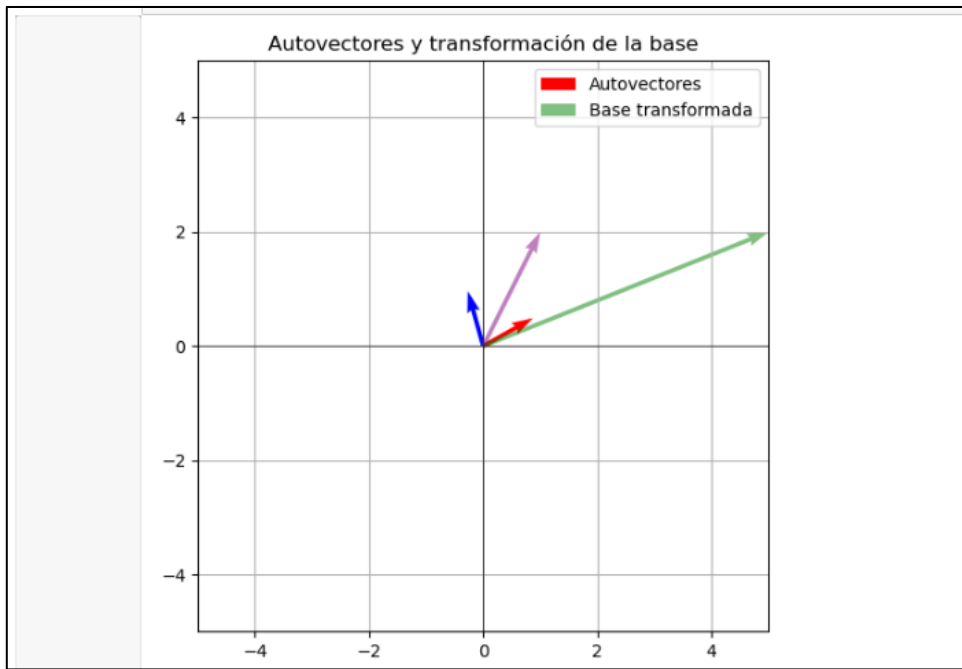
# Graficar autovectores
plt.quiver(*origin, eigenvectors[0], eigenvectors[1],
          angles='xy', scale_units='xy', scale=1,
          color=['r', 'b'], label='Autovectores')

# Graficar transformación de la base estándar
x = np.array([[1, 0], [0, 1]])
transformed = A @ x.T
plt.quiver(*origin, transformed[0], transformed[1],
          angles='xy', scale_units='xy', scale=1,
          color=['g', 'purple'], alpha=0.5, label='Base transformada')

# Estética del gráfico
plt.xlim(-5, 5)
plt.ylim(-5, 5)
plt.grid()
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.title("Autovectores y transformación de la base")
plt.gca().set_aspect('equal')
plt.legend()

# Guardar imagen
plt.savefig("autovectores_matriz_A.png", dpi=300)
plt.show()
```

En la siguiente página, graficamos los resultados obtenidos en un plano con Matplotlib.



Las flechas rojas y azules representan los autovectores, que mantienen su dirección bajo la acción de A .

Las flechas verdes y moradas muestran cómo la base estándar se transforma, evidenciando el efecto de la matriz sobre el espacio.

Esta visualización confirma que los autovectores son direcciones privilegiadas donde la transformación solo escala, sin rotar.

Conclusiones

Las funciones matemáticas son herramientas que nos permiten modelar fenómenos del mundo real. Ya sea en física o en campos como ingeniería, economía o inteligencia artificial (IA), permiten representar relaciones entre variables y analizar el comportamiento de sistemas complejos. Algunos conceptos de interés para la IA son:

Gradiente: Vector que apunta en la dirección de máximo incremento de una función multivariable.

Derivada: Medida del cambio instantáneo en funciones de una sola variable, representa la tangente de la función en un punto determinado.

El gradiente y la derivada son operadores que nos sirven para analizar funciones en puntos específicos. Nos permiten obtener información sobre la dirección de mayor crecimiento, la tasa de cambio y la sensibilidad de los sistemas modelados.

Funciones Vectoriales:

Cuando el número de variables crece, es conveniente expresar las funciones en forma vectorial. Esto permite aplicar herramientas del álgebra lineal como:

- Matrices Jacobianas
- Transformaciones lineales
- Espacios vectoriales y normas

Propiedades de la Diferenciación:

Entre las propiedades más relevantes se encuentran:

- Regla del producto
- Regla de la suma
- Regla de la cadena, esencial en algoritmos de aprendizaje automático como el backpropagation.

En el aprendizaje automático, la diferenciación es una herramienta que nos permite entrenar modelos, ajustar sus parámetros y optimizar funciones de pérdida.

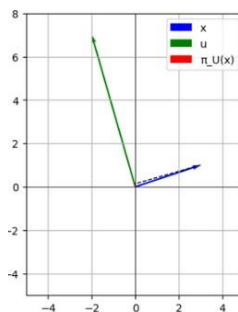
Liga al código en Github

<https://github.com/luisgg121/ML-Actividad-02.git>

Anexo.- Infografía sobre Tipos de Modelos Vectoriales y sus aplicaciones

Exploración de Tipos de Modelos Vectoriales y sus Aplicaciones

Fundamentos matemáticos y usos en análisis y reconocimiento



Definición y fundamentos de los modelos vectoriales

Representación mediante vectores

Los modelos vectoriales usan vectores en espacios matemáticos para representar entidades de forma precisa y estructurada.

Cuantificación de relaciones

Permiten medir las similitudes y diferencias entre datos para facilitar análisis precisos y comparativos.

Análisis estructurado de datos

Facilitan la manipulación y análisis de datos complejos en múltiples áreas mediante su estructura matemática.

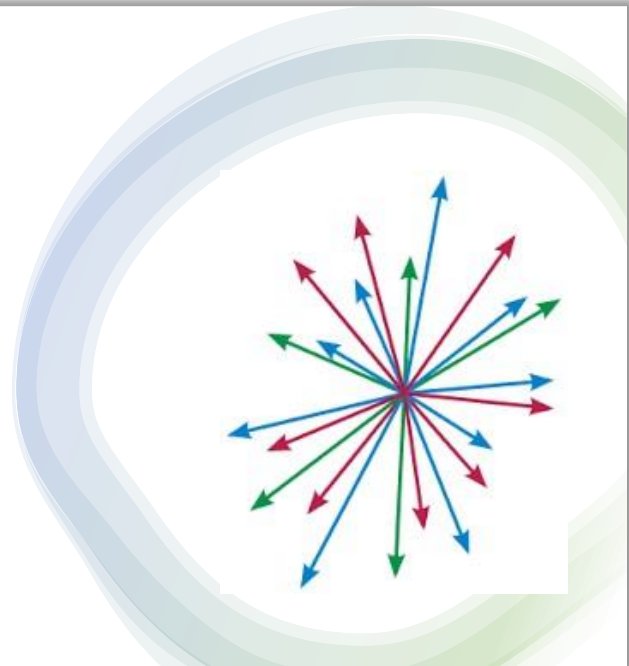
Modelo de espacio vectorial (VSM)

Representación Vectorial

Documentos y consultas se representan como vectores en un espacio multidimensional para análisis eficiente.

Recuperación de Información

El modelo facilita la recuperación y comparación de información basada en la similitud entre vectores.



Importancia de los modelos de vectores en la ciencia y tecnología

Modelado de fenómenos complejos

Los modelos de vectores ayudan a representar fenómenos complejos en física y otras ciencias con precisión matemática.

Procesamiento de grandes datos

Permiten procesar y analizar grandes volúmenes de datos en informática e ingeniería eficientemente.

Mejora de inteligencia artificial

Los modelos vectoriales son clave para optimizar algoritmos avanzados de inteligencia artificial y aprendizaje automático.



Definición y características principales de los modelos de vectores



$$= V \cos \theta$$

Teorema de Pitágor:

$$|r| = \sqrt{Vx^2 + Vy^2}$$



Representación de datos

Los modelos de vectores usan magnitudes y direcciones para representar datos en espacios multidimensionales.

Captura de relaciones complejas

Permiten captar relaciones complejas entre datos usando la dirección y magnitud de los vectores.

Operaciones matemáticas precisas

Facilitan operaciones matemáticas exactas para análisis y procesamiento de datos.

Importancia en matemáticas y ciencias aplicadas

Modelos vectoriales en matemáticas

Los modelos vectoriales son fundamentales para el álgebra lineal y la geometría analítica en matemáticas.

Aplicaciones en ciencias aplicadas

Se emplean para modelar fenómenos, procesar señales y desarrollar algoritmos en inteligencia artificial y análisis computacional.



Física: representación de magnitudes y fuerzas

Importancia de los vectores

Los vectores representan magnitudes físicas que tienen dirección y sentido, fundamentales para describir fenómenos físicos.

Aplicación en fuerzas

Las fuerzas se representan como vectores para analizar su magnitud y dirección en diferentes situaciones físicas.

Análisis y predicción

El uso de vectores facilita el análisis y la predicción del comportamiento de sistemas físicos complejos.

Concepto del espacio euclidiano

Definición del espacio euclidiano

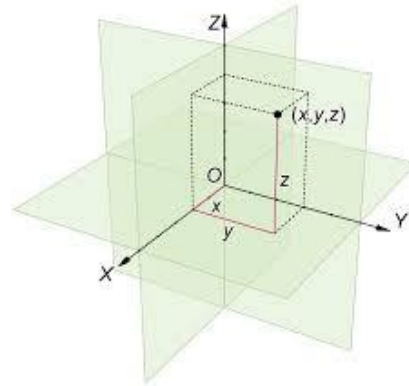
El espacio euclidiano es un espacio vectorial con una métrica basada en la distancia euclidiana para medir distancias lineales.

Representación con coordenadas

Permite representar puntos y objetos mediante coordenadas en un sistema cartesiano para facilitar cálculos geométricos.

Aplicaciones geométricas y analíticas

El espacio euclidiano forma la base para muchas aplicaciones en geometría y análisis matemático.



Cálculo de distancias euclidianas

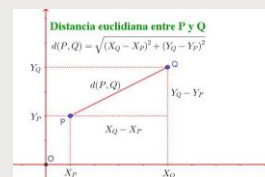
Definición matemática

La distancia euclidiana es la raíz cuadrada de la suma de las diferencias al cuadrado entre coordenadas.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

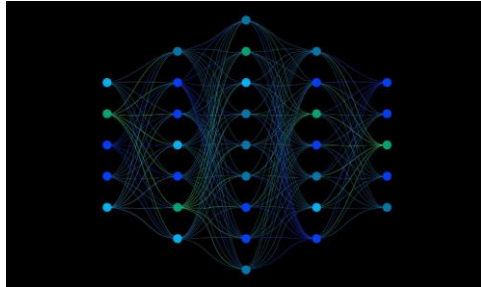
Interpretación geométrica

Representa la distancia recta o mínima entre dos puntos en un espacio multidimensional.



Aplicación en espacios multidimensionales

Es una medida intuitiva de proximidad en espacios con múltiples dimensiones o variables.



Aplicaciones en geometría y análisis de datos

Resolución de problemas geométricos

El modelo ayuda a resolver problemas clásicos y modernos en geometría con precisión y eficiencia.

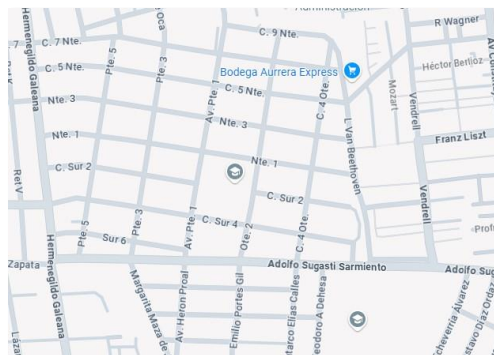
Análisis de datos y clustering

Se usa para agrupar datos similares facilitando la identificación de patrones y tendencias en grandes conjuntos de datos.

Clasificación y visualización

El modelo permite clasificar datos y visualizar información compleja para facilitar su interpretación y toma de decisiones.

Modelo de distancia de Manhattan (taxicab)



Definición de distancia Manhattan

La distancia Manhattan suma las diferencias absolutas entre las coordenadas de dos puntos.

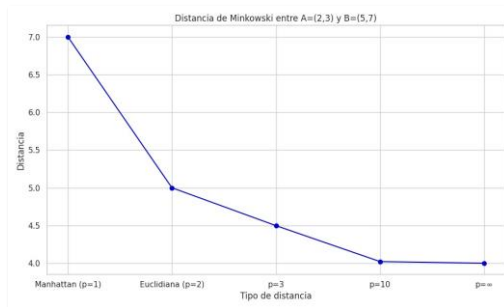
Aplicación en entornos urbanos

Este modelo es útil para medir distancias en ciudades con calles en cuadrícula y rutas ortogonales.

Restricción de movimiento

Considera movimientos limitados a rutas verticales y horizontales, no diagonales.

Modelo de distancia de Minkowski



La distancia de Minkowski es una fórmula matemática que permite medir la distancia entre dos puntos en un espacio multidimensional, ajustando la forma del cálculo mediante un parámetro p . Es una generalización de otras métricas conocidas como la distancia euclidiana (cuando $p=2$) y la distancia Manhattan (cuando $p=1$).

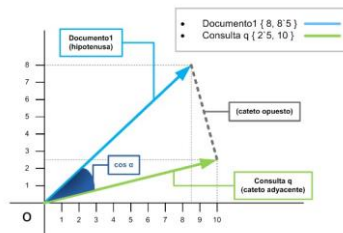
Parámetro p ajustable

El parámetro "p" permite adaptar la métrica a diferentes problemas optimizando la medida de distancia.

Usos:

- En machine learning, especialmente en algoritmos como k-NN y K-means, para medir similitud entre datos.
- En análisis de datos, para detectar anomalías o agrupar elementos similares.
- En espacios vectoriales normados, donde se requiere flexibilidad para adaptar la métrica a la geometría del problema.

Modelos vectoriales ponderados y sus variaciones



Asignación de pesos a vectores

Los modelos ponderados asignan diferentes pesos para reflejar la importancia relativa de cada característica vectorial.

Precisión mejorada

El uso de pesos mejora la precisión y relevancia en tareas como búsqueda y análisis de datos.

Uso en recuperación de información y búsqueda de texto

Modelos Vectoriales

Representan documentos y consultas como vectores matemáticos para facilitar la comparación.

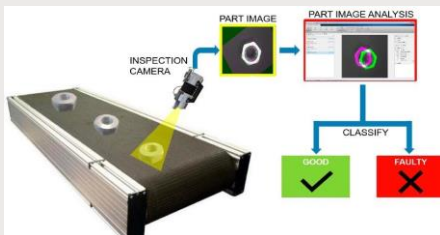
Comparación y Recuperación

Permiten comparar eficientemente vectores para recuperar información relevante en grandes bases de datos.

$$\vec{Doc}_{(d)} = (P_{(1,d)}, P_{(2,d)}, P_{(3,d)}, P_{(n,d)})$$

Diagram illustrating the vector representation of a document. The vector $\vec{Doc}_{(d)}$ is composed of terms $P_{(1,d)}, P_{(2,d)}, P_{(3,d)}, P_{(n,d)}$. Callouts explain the components:
 - **Peso del término** (Term weight) points to the term $P_{(2,d)}$.
 - **Identificador del documento; documento1, documento2,...** (Document identifier) points to the subscript (d) .
 - **Identificador del término por su posición en el fichero diccionario; término1, término2, término3,...** (Term identifier) points to the subscript (n) in $P_{(n,d)}$.

Aplicaciones en reconocimiento de patrones y machine learning



Modelos vectoriales

Los modelos vectoriales representan datos para facilitar el análisis y la identificación precisa de patrones en aprendizaje automático.

Identificación de patrones

Los algoritmos entrenados con modelos vectoriales detectan patrones complejos para clasificar y predecir con alta precisión.

Clasificación y predicción

Los sistemas de machine learning usan características vectoriales para clasificar datos y hacer predicciones confiables en diversos campos.

Implementación en sistemas de recomendación y análisis de similitud



Evaluación de similitud

Los modelos analizan la similitud entre usuarios o ítems para identificar preferencias comunes y patrones.

Recomendaciones personalizadas

Se generan recomendaciones adaptadas para mejorar la experiencia individual en plataformas digitales.

Aplicaciones comerciales

Estos sistemas se aplican en plataformas comerciales para aumentar la satisfacción y fidelización del cliente.

Conclusión

Importancia de modelos vectoriales

Los modelos vectoriales son esenciales para representar datos complejos en varios formatos y disciplinas científicas.

Aplicaciones prácticas

Estos modelos se emplean en análisis avanzados, ciencia y tecnología para facilitar la comprensión de información diversa.